Local Agents with Hermes — Cheatsheet

Hermes Agent v0.21 · every concept mapped to its Claude Code equivalent · the model is a component, the context is the asset

THE MENTAL MAP

Layer	Is	Role
Your agent	instructions + memory + skills + tools	the durable asset
Harness	Hermes / Claude Code / OpenClaw	loop, tools, safety
Model	Qwen3.5-9B / Qwen3.6-35B-A3B / Hermes 4.3	swappable

No weights change anywhere in this course. "Self-improves through use" is context engineering, not fine-tuning.

INSTALL & FIRST RUN

```
curl -fsSL https://hermes-agent.nousresearch.com/
install.sh | bash
hermes --version
hermes model      # Custom endpoint: base URL + API
key
hermes --tui
```

`pip install hermes-agent` lags the installer (PyPI 0.19.0 vs v0.21.x) and skips bundled Node/ripgrep/ffmpeg. Use the installer.

PICK A LOCAL MODEL

Ask in order: **RAM budget? → task? → is tool-calling reliable?**

Model	RAM	Fit
Qwen3.5-9B	8 GB+ (64K) · ~10 GB (128K)	Official Mac starter
Qwen3.6-35B-A3B	32 GB+	Daily driver
Hermes 4.3 36B	32 GB+	Nous, tuned for tools

≥64K context or nothing. Mac path: llama.cpp or omlx/MLX behind a Custom endpoint (Ollama also works).

```
llama-server -m qwen3.5-9b-q4.gguf -c 131072 \
  --cache-type-k q4_0 --cache-type-v q4_0 --port 8080
# 8 GB Mac: -c 65536   → http://localhost:8080/v1
```

Local model "doesn't work"? Suspect the **tool-call parser** first.

WHERE THINGS LIVE

Purpose	Hermes	Claude Code
Project rules	`AGENTS.md` (repo root)	`CLAUDE.md`
Personality	`~/.hermes/SOUL.md`	`~/.claude/CLAUDE.md`
Learned facts	`MEMORY.md`, `USER.md`	Auto Memory
Procedures	`~/.hermes/skills/*/SKILL.md`	`~/.claude/skills/`
Config	`~/.hermes/config.yaml`, `.env`	`settings.json`

Context files sit in the cached system prompt, so editing them mid-session breaks the cache.

THE START-SAFE TOOL LADDER

Rung	Enable	Worst case
1 Read-only	filesystem read, ripgrep	wasted tokens
2 Draft-only	new scratch files; `execute_code` off	a junk file
3 Reversible	edits inside a git repo	`git checkout .`
4 Irreversible	delete, terminal, send, spend, prod	you explain it

Tools, MCP servers, and cron jobs all climb the same ladder. Approvals: **once** / **session** / **always** / **deny**, so pick **session** until you trust it. Default `approvals.mode: smart`.

THE 60-SECOND EVAL HABIT

25 real tasks in `assets/eval-template.csv`. Score only **right tool?** and **right args?**

- Below ~80% tool accuracy, the harness will feel broken.
- Same 25 rows compare two models objectively.
- Re-run after every new skill and every MCP server.

MEMORY

- `MEMORY.md` ~2,200 chars · `USER.md` ~1,375 chars: small on purpose, in every turn's prompt.
- **Frozen mid-session:** a memory write lands on disk now but reaches the prompt **next session**.
- `session_search`: FTS5 over **all** past raw sessions.
- Curator LLM-consolidation is **off** by default; skill stale → archived moves run automatically.

```
remember this for next time: the staging DB is at db-
staging.internal:5432
```

SKILLS

Reference document → skill; sticky note → memory. Promote a task to a skill when it's **5+ steps and repeats**. Same agentskills.io `SKILL.md` standard as Claude Code, so skills are portable both ways.

1. name + description, always loaded (~100–200 tokens)
2. `SKILL.md` body, on trigger (<~500 lines)
3. reference files, on demand

```
save what you just did as a skill called deploy-
staging
/skills · /skills browse · hermes skills install
official/<name>
```

Fixed deterministic sequence? Write a script, not a skill.

THE SELF-IMPROVEMENT LADDER

Rung	Behavior
1	Writes a file if told
2	User-triggered persistence (`CLAUDE.md`, `AGENTS.md`)
3	Decides what to remember, unprompted (Claude Code reaches here)
4	+ unattended long-horizon curation; retrieval by search
5	+ writes *and retires* its own skills

Hermes ships rungs 3→5 on by default. **The differentiator is default integration, not intelligence.**

MCP

```
mcp_servers:                  # ~/.hermes/config.yaml
  filesystem:
    command: npx
    args: ["-y", "@modelcontextprotocol/server-
filesystem", "/Users/you/projects"]
    tools:
      include: ["read_text_file", "list_directory"]
  fetch:
    command: uvx
    args: ["mcp-server-fetch"]
```

Tools register as `mcp_<server>_<tool>`. `tools.include` is a **security control**. `hermes mcp` · `/reload-mcp` · `hermes mcp serve` exposes a messaging bridge to Claude Code.

GATEWAY & CRON

```
hermes gateway setup && hermes gateway install &&
hermes gateway
/sethome          # where cron output lands; set it
first
/cron add "0 8 * * 1-5" "Check ~/projects/api for PRs
opened since
  yesterday... If none, reply [SILENT]." --name
morning-prs --deliver telegram
```

- Cron runs in **fresh sessions**, so the prompt must carry all context.
- `--script` preflight · `--skill` attach · `--deliver local` first · `[SILENT]` mutes "all fine".
- `/cron run <name>` once before trusting a schedule.

COST: THE PROMPT CACHE

A stable system prompt earns cache hits. **Cache breakers:** `/model` switches, provider fallbacks, credential rotation, editing context files mid-session.

`/compress` · `/usage` · `hermes prompt-size` · `/insights` · fan out with `delegate_task`, batch with `execute_code`.

SECURITY CHECKLIST

> **Container backends skip the dangerous-command check.**
> With `TERMINAL_ENV=docker`, the image is your whole defense.

- Never `GATEWAY_ALLOW_ALL_USERS=true` on a terminal bot. Use `*_ALLOWED_USERS`.
- Secrets in `~/.hermes/.env`, never `config.yaml` or `AGENTS.md`.
- `chmod 700 ~/.hermes/whatsapp-session`: live credentials.
- MCP tool descriptions are a prompt-injection surface. Read what you install.

WHICH HARNESS?

Use	When
Hermes	Persistent workflows, learning from work, one agent across CLI + messaging
Claude Code + Ollama	Deep IDE integration, most mature tooling, focused coding
OpenClaw	Lightest setup, pure CLI coding

Pick Hermes when the accumulated context is the point; pick a stateless coding agent when each session is disposable.

Build the context deliberately, wire new power at low blast radius first, and measure with an eval set instead of a feeling.